2025
2026

Projet De Synthèse

EFREI M1-CSAI - CLOUD COMPUTING

STÉPHANE LARCHER

PROJET TECHNIQUE

Déploiement d'une Application Cloud-Native
avec Docker, Kubernetes et Terraform

Informations Clés

Travail en groupe de 3 étudiants maximum
Coefficient : 50% de la note des modules concernés
Livrables : Dépôt Git + Documentation technique + Soutenance orale (15 min)

Année universitaire 2025-2026

Table des matières

1. Contexte et Objectifs *3*
1.1 Contexte Pédagogique 3
1.2 Compétences Techniques Évaluées 3
2. Sujet du Projet : Plateforme E-Commerce *4*
2.1 Présentation de l'Application 4
Architecture Applicative 4
2.2 Exigences Non-Fonctionnelles 4
3. Travail Demandé *5*
3.1 Phase 1 : Conteneurisation (Docker) 5
Objectifs 5
Livrables Phase 1 5
3.2 Phase 2 : Orchestration (Kubernetes) 5
Objectifs 5
Livrables Phase 2 5
3.3 Phase 3 : Infrastructure as Code (Terraform) 5
Objectifs 5
Livrables Phase 3 5
3.4 Phase 4 : CI/CD et Observabilité 5
Objectifs 6
Livrables Phase 4 6
4. Livrables Globaux *7*
4.1 Dépôt Git 7
4.2 Documentation Technique (15-20 pages) 7
5. Environnement Technique *8*
5.1 Plateformes Autorisées 8
5.2 Outils Requis 8

1. Contexte et Objectifs

1.1 Contexte Pédagogique

Ce projet constitue l'évaluation technique des modules avancés du cours de Cloud Computing. Il vise à évaluer votre capacité à concevoir, déployer et opérer une infrastructure cloud moderne en utilisant les outils et pratiques DevOps actuels.

Vous devrez démontrer votre maîtrise de la conteneurisation avec Docker, de l'orchestration avec Kubernetes, de l'Infrastructure as Code avec Terraform, ainsi que votre capacité à mettre en œuvre des pipelines CI/CD.

1.2 Compétences Techniques Évaluées

- Conteneuriser une application multi-services avec Docker et Docker Compose
- Rédiger des Dockerfiles optimisés (multi-stage builds, sécurité, taille d'image)
- Déployer et orchestrer des applications sur un cluster Kubernetes
- Écrire des manifestes Kubernetes (Deployments, Services, ConfigMaps, Secrets, Ingress)
- Provisionner une infrastructure cloud avec Terraform (ou équivalent IaC)
- Implémenter un pipeline CI/CD automatisé (GitHub Actions, GitLab CI, ou équivalent)
- Mettre en place des stratégies de monitoring et de logging
- Appliquer les bonnes pratiques de sécurité (RBAC, Network Policies, gestion des secrets)

2. Sujet du Projet : Plateforme E-Commerce

2.1 Présentation de l'Application

TECHSHOP - Application E-Commerce Microservices

Application de démonstration à déployer en production

TechShop est une application e-commerce construite selon une architecture microservices. Elle servira de base pour démontrer vos compétences en déploiement cloud-native. L'application se compose des services suivants :

Architecture Applicative

Service	Description	Technologie	Port
Frontend	Interface utilisateur React	React / Nginx	80
API Gateway	Point d'entrée API, routage	Node.js / Express	3000
User Service	Gestion des utilisateurs, auth	Python / FastAPI	8001
Product Service	Catalogue produits	Java / Spring Boot	8002
Order Service	Gestion des commandes	Go	8003
Database	Base de données principale	PostgreSQL	5432
Cache	Cache en mémoire	Redis	6379
Message Queue	Communication asynchrone	RabbitMQ	5672

Note : Le code source de l'application vous sera fourni. Votre travail consiste à conteneuriser, orchestrer et déployer cette application, et non à la développer.

2.2 Exigences Non-Fonctionnelles

- Haute disponibilité : l'application doit supporter la panne d'un nœud
- Scalabilité : les services doivent pouvoir scaler horizontalement
- Sécurité : communications chiffrées, gestion des secrets, principe du moindre privilège
- Observabilité : métriques, logs centralisés, health checks
- Déploiement automatisé : mise à jour sans interruption de service (rolling updates)

3. Travail Demandé

3.1 Phase 1 : Conteneurisation (Docker)

Objectifs

1. Créer un Dockerfile optimisé pour chaque service applicatif
2. Implémenter des multi-stage builds pour réduire la taille des images
3. Configurer un fichier docker-compose.yml pour le développement local
4. Mettre en place un registry privé ou utiliser un registry cloud

Livrables Phase 1

- Dockerfiles pour chaque service avec commentaires explicatifs
- Fichier docker-compose.yml fonctionnel
- Documentation des choix d'images de base et des optimisations
- Scripts de build et de push vers le registry

3.2 Phase 2 : Orchestration (Kubernetes)

Objectifs

1. Écrire les manifestes Kubernetes pour chaque composant
2. Configurer les ressources : Deployments, Services, ConfigMaps, Secrets
3. Mettre en place un Ingress Controller avec certificat TLS
4. Implémenter des Horizontal Pod Autoscalers (HPA)
5. Configurer des Network Policies pour la sécurité réseau
6. Gérer la persistance des données avec PersistentVolumeClaims

Livrables Phase 2

- Manifestes Kubernetes organisés (structure Kustomize ou Helm charts)
- Configuration des namespaces et RBAC
- Documentation de l'architecture Kubernetes déployée
- Procédures de déploiement et de rollback

3.3 Phase 3 : Infrastructure as Code (Terraform)

Objectifs

1. Provisionner l'infrastructure cloud nécessaire (cluster K8s, réseau, stockage)
2. Structurer le code Terraform en modules réutilisables
3. Gérer l'état Terraform de manière sécurisée (remote state)
4. Implémenter des environnements multiples (dev, staging, prod)

Livrables Phase 3

- Code Terraform modulaire et documenté
- Configuration du backend remote state
- Fichiers tfvars pour chaque environnement
- Documentation des ressources provisionnées

3.4 Phase 4 : CI/CD et Observabilité

Objectifs

1. Créer un pipeline CI/CD complet (build, test, scan, deploy)
2. Intégrer des tests automatisés et une analyse de sécurité des images
3. Mettre en place un stack de monitoring (Prometheus/Grafana ou équivalent)
4. Configurer la centralisation des logs (ELK/Loki ou équivalent)

Livrables Phase 4

- Fichiers de configuration CI/CD (.github/workflows ou .gitlab-ci.yml)
- Dashboard Grafana avec les métriques clés
- Configuration des alertes
- Documentation du pipeline et des procédures d'incident

4. Livrables Globaux

4.1 Dépôt Git

Structure attendue du repository :

```
techshop-infrastructure/
├── docker/
│   ├── frontend/Dockerfile
│   ├── api-gateway/Dockerfile
│   ├── user-service/Dockerfile
│   └── ...
├── kubernetes/
│   ├── base/
│   ├── overlays/
│   └── helm-charts/
├── terraform/
│   ├── modules/
│   ├── environments/
│   └── backend.tf
├── .github/workflows/ (ou .gitlab-ci.yml)
├── monitoring/
├── docs/
└── README.md
```

4.2 Documentation Technique (15-20 pages)

Section	Contenu Attendu
1. Introduction	Présentation du projet, objectifs, périmètre technique
2. Architecture	Diagrammes d'architecture (applicative, infrastructure, réseau), choix techniques justifiés
3. Conteneurisation	Stratégie de build, optimisations, gestion des images, sécurité des conteneurs
4. Orchestration K8s	Description des ressources, stratégie de déploiement, scaling, networking
5. Infrastructure	Ressources Terraform, gestion des états, approche multi-environnements
6. CI/CD	Description du pipeline, étapes, tests, stratégie de déploiement
7. Observabilité	Stack de monitoring, métriques clés, alertes, dashboards
8. Sécurité	Mesures de sécurité appliquées, gestion des secrets, RBAC, Network Policies
9. Conclusion	Bilan, difficultés rencontrées, améliorations possibles

5. Environnement Technique

5.1 Plateformes Autorisées

Vous pouvez utiliser l'une des plateformes suivantes pour le déploiement :

- Minikube ou Kind pour le développement local
- Google Kubernetes Engine (GKE) - crédits éducatifs disponibles
- Amazon Elastic Kubernetes Service (EKS)
- Azure Kubernetes Service (AKS)
- DigitalOcean Kubernetes
- OVHcloud Managed Kubernetes

5.2 Outils Requis

Catégorie	Outil	Version minimale
Conteneurisation	Docker, Docker Compose	Docker 24+, Compose v2
Orchestration	kubectl, Helm (optionnel)	kubectl 1.28+, Helm 3.x
IaC	Terraform ou OpenTofu	Terraform 1.5+
CI/CD	GitHub Actions ou GitLab CI	-
Monitoring	Prometheus, Grafana	Prometheus 2.x, Grafana 10+
Logging	Loki ou ELK Stack	-